REPORTE DE PRÁCTICA NO. 1

Práctica 1. Análisis de Expresiones Regulares

ALUMNO:
Arrazola Ibarra Juan Pablo

Introducción

Una **Gramática Formal** definida sobre un alfabeto Σ es una tupla de la forma:

$$G = \{\Sigma_T, \Sigma_N, S, P\}$$

donde:

- Σ_T es un alfabeto de símbolos terminales
- Σ_N es un alfabeto de símbolos no terminales
- S es el símbolo inicial de la gramática
- P es un conjunto de producciones gramaticales

Además, se cumple:

$$S \in \Sigma_N$$

$$\Sigma_T \cap \Sigma_N =$$

$$\Sigma = \Sigma_T \cup \Sigma_N$$

La gramática formal G permite generar un lenguaje:

$$L = \{x \in \Sigma_T^* \mid S \Rightarrow^* x\}$$

Por lo tanto, las palabras del lenguaje estarán formadas por cadenas de símbolos terminales generadas a partir del símbolo inicial de la gramática utilizando las producciones que la definen.
Una **expresión regular** es una notación normalizada para representar lenguajes regulares, es decir, lenguajes generados por gramáticas regulares (Tipo 3). Las expresiones regulares permiten describir con exactitud y sencillez cualquier lenguaje regular.

Marco teórico

El análisis de cadenas de símbolos es un tema fundamental en el área de las ciencias computacionales, ya que permite interpretar y procesar información que está representada en forma de texto. Este tipo de análisis es utilizado en diferentes áreas como los compiladores, los intérpretes, el procesamiento de lenguaje natural y la validación de datos. En esta práctica se trabaja con cadenas de símbolos para identificar distintos tipos de palabras dentro de un lenguaje definido.

Un concepto importante es el **alfabeto de símbolos**, el cual se define como el conjunto de caracteres permitidos dentro de un lenguaje. A partir de un alfabeto se pueden formar cadenas de símbolos, que son secuencias de caracteres que siguen ciertas reglas. Estas reglas determinan qué cadenas son válidas dentro de un lenguaje y cuáles no lo son.

Las **expresiones regulares** son una herramienta que permite describir patrones que deben cumplir las cadenas de símbolos. Mediante las expresiones regulares es posible identificar números enteros, palabras en minúsculas, palabras en mayúsculas, identificadores y símbolos. Su uso facilita la validación y clasificación de textos de forma automática dentro de un programa.

Por otro lado, las **estructuras de datos** permiten almacenar y organizar la información que se obtiene durante el análisis de las cadenas. En esta práctica se utilizan estructuras simples como arreglos y contadores para guardar las palabras encontradas en cada categoría y llevar un registro de cuántos elementos pertenecen a cada tipo.

Finalmente, el **proceso de análisis de cadenas** consiste en tomar una cadena de entrada, dividirla en partes más pequeñas (tokens) y evaluar cada una según las reglas definidas. Este procedimiento permite clasificar la información de manera ordenada y obtener resultados que pueden ser utilizados para posteriores análisis o validaciones dentro de un sistema.

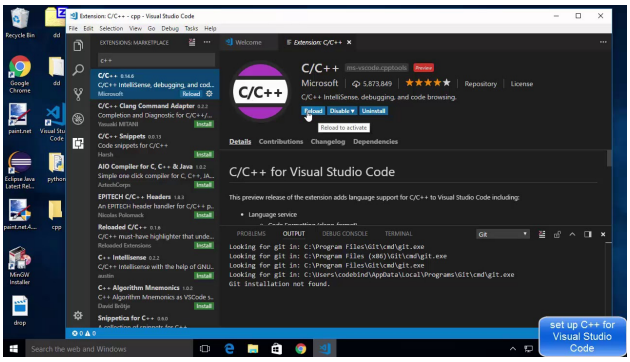
Objetivo general

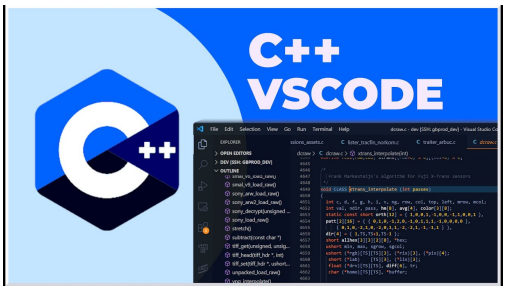
Construir programas en un lenguaje de programación de alto nivel que realicen el análisis de cadenas de símbolos que forman palabras, basado en la implementación de estructuras de datos y algoritmos de análisis, además del uso de expresiones regulares como segunda estrategia de implementación.

Objetivos específicos

- Identificar el proceso básico de análisis de cadenas de símbolos de un alfabeto.
- Implementar estructuras de datos y algoritmos de análisis de cadenas de símbolos en un lenguaje de programación de alto nivel.
- Implementar expresiones regulares para el análisis de cadenas de símbolos en un lenguaje de programación de alto nivel.

2. Herramientas empleadas



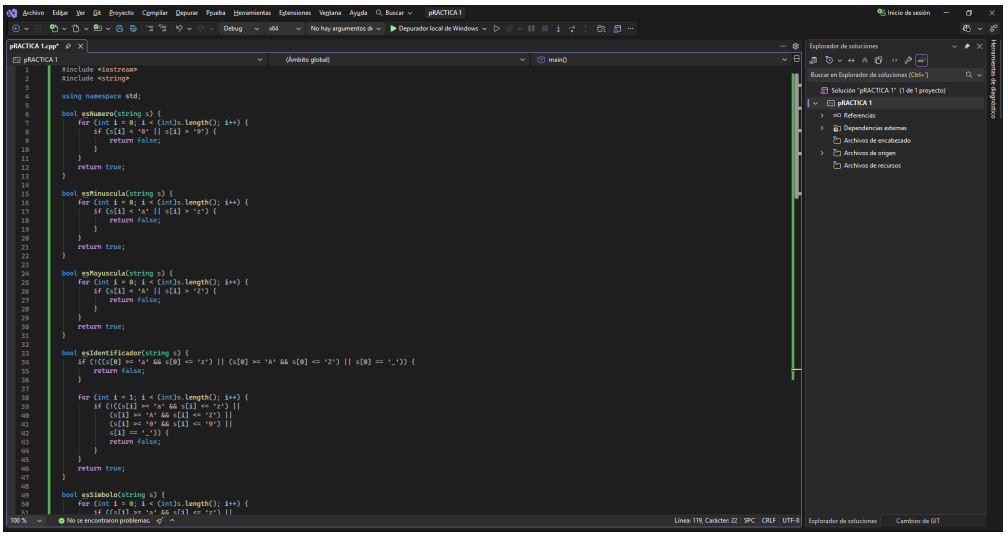


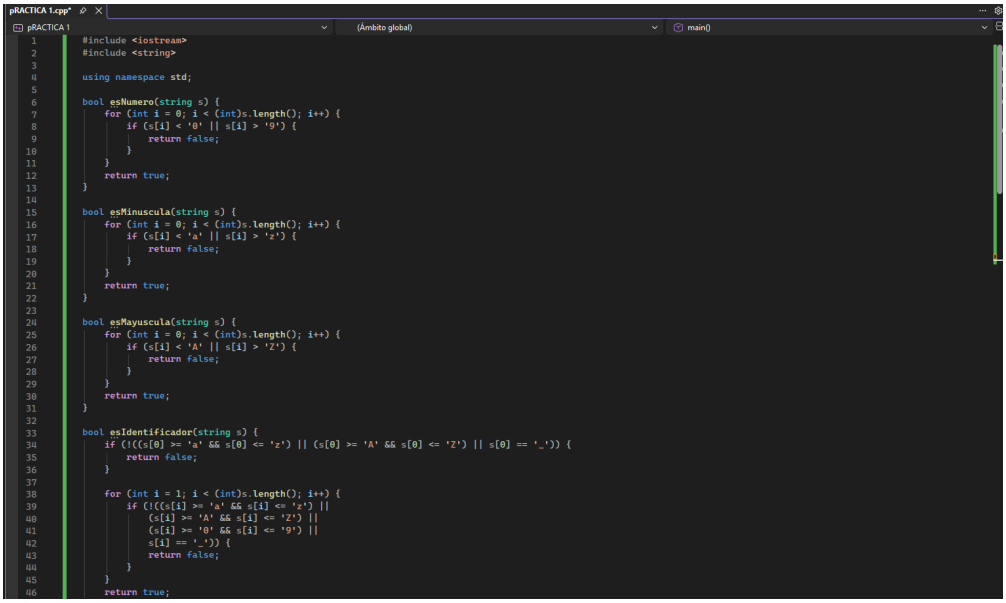
Las imágenes anteriores corresponden a las herramientas empleadas durante el desarrollo de la práctica. Se utilizó el entorno de desarrollo Visual Studio para la implementación del programa en el lenguaje de programación C++, el cual permitió la compilación, ejecución y depuración del analizador de cadenas de símbolos solicitado.

Desarrollo de la práctica

1. Utilizar un lenguaje de programación para construir programas basados en estructuras de datos para analizar cadenas de símbolos con el propósito de identificar:
   - Número entero
   - Palabras en minúsculas
   - Palabras en mayúsculas
   - Identificadores (nombres de variables)
2. Definir el alfabeto de símbolos y las expresiones regulares que permitan identificar los siguientes tipos de palabras de un lenguaje regular:
   - Números enteros
   - Palabras en minúsculas
   - Palabras en mayúsculas
   - Identificador (nombres de variables)
   - Símbolos
3. Utilizar un lenguaje de programación para construir programas basados en expresiones regulares para analizar cadenas de símbolos con el propósito de clasificar y contabilizar las palabras en las categorías:
   - Número entero
   - Palabras en minúsculas
   - Palabras en mayúsculas
   - Identificador (nombre de variable)
   - Símbolo

Evidencias





pRACTICA 1.cpp*

pRACTICA 1 | (Ámbito global) | main()

```cpp
        return true;
    }

    bool esSimbolo(string s) {
        for (int i = 0; i < (int)s.length(); i++) {
            if ((s[i] >= 'a' && s[i] <= 'z') ||
                (s[i] >= 'A' && s[i] <= 'Z') ||
                (s[i] >= '0' && s[i] <= '9') ||
                s[i] == '_') {
                return false;
            }
        }
        return true;
    }

    int main() {
        cout << "Analizador de cadenas de simbolos" << endl;
        cout << "Escribe la cadena: ";

        string cadena;
        getline(cin, cadena);

        string palabra = "";

        int contNumeros = 0;
        int contMinusculas = 0;
        int contMayusculas = 0;
        int contIdentificadores = 0;
        int contSimbolos = 0;

        cout << "\nResultados:\n";

        for (int i = 0; i <= (int)cadena.length(); i++) {

            if (i == (int)cadena.length() || cadena[i] == ' ') {

                if (palabra != "") {
                    if (esNumero(palabra)) {
                        cout << "Numero: " << palabra << endl;
                        contNumeros++;
                    }
                    else if (esMinuscula(palabra)) {
                        cout << "Minuscula: " << palabra << endl;
                        contMinusculas++;
                    }
                    else if (esMayuscula(palabra)) {
```

pRACTICA 1.cpp*

pRACTICA 1 | (Ámbito global) | main()

```cpp
                if (palabra != "") {
                    if (esNumero(palabra)) {
                        cout << "Numero: " << palabra << endl;
                        contNumeros++;
                    }
                    else if (esMinuscula(palabra)) {
                        cout << "Minuscula: " << palabra << endl;
                        contMinusculas++;
                    }
                    else if (esMayuscula(palabra)) {
                        cout << "Mayuscula: " << palabra << endl;
                        contMayusculas++;
                    }
                    else if (esIdentificador(palabra)) {
                        cout << "Identificador: " << palabra << endl;
                        contIdentificadores++;
                    }
                    else if (esSimbolo(palabra)) {
                        cout << "Simbolo: " << palabra << endl;
                        contSimbolos++;
                    }
                }

                palabra = "";
            }
            else {
                palabra = palabra + cadena[i];
            }
        }

        cout << "\n--- Conteo final ---\n";
        cout << "Numeros: " << contNumeros << endl;
        cout << "Minusculas: " << contMinusculas << endl;
        cout << "Mayusculas: " << contMayusculas << endl;
        cout << "Identificadores: " << contIdentificadores << endl;
        cout << "Simbolos: " << contSimbolos << endl;

        system("pause");
        return 0;
    }
```

```
Analizador de cadenas de simbolos
Escribe la cadena: Hola profesor Como esta?

Resultados:
Identificador: Hola
Minuscula: profesor
Identificador: Como

--- Conteo final ---
Numeros: 0
Minusculas: 1
Mayusculas: 0
Identificadores: 2
Simbolos: 0
Presione una tecla para continuar . . .
```

Repositorio del código fuente

Como parte de la evidencia del desarrollo de la práctica, se incluye el enlace permanente al repositorio donde se encuentra el código fuente del programa implementado:

https://github.com/Dextrela/Automatas-Y-Compiladores-Arrazola-Ibarra-Juan-Pablo/blob/d774f72610020287b5df59f17dc3d08207b91a00/Practica%201

Cuestionario

- **¿Cuál es la utilidad de las expresiones regulares en la identificación de palabras de un lenguaje?**

  Las expresiones regulares sirven para poder reconocer patrones dentro de un texto. En esta práctica se usaron para identificar qué tipo de palabra era cada parte de la cadena que ingresaba el usuario, por ejemplo, si era un número entero, una palabra en minúsculas, en mayúsculas o un identificador. Esto facilita mucho el análisis porque el programa puede clasificar automáticamente cada palabra.

- **¿Cuáles elementos del lenguaje de programación utilizaste para la implementación de las estructuras de datos?, ¿cómo las utilizaste?**

  Se utilizaron arreglos y contadores para guardar la información obtenida. Los arreglos se usaron para almacenar las palabras que pertenecen a cada categoría y los contadores para saber cuántas palabras de cada tipo se encontraron durante el análisis de la cadena de símbolos.

- **¿Cuál es el proceso que seguiste para analizar las cadenas de símbolos?**

  Primero el programa pide al usuario que ingrese una cadena de texto. Después, esa cadena se separa en partes usando los espacios como separador. Cada palabra se revisa con las expresiones regulares para saber a qué categoría pertenece. Al final, el programa muestra en pantalla los resultados de la clasificación.

- **¿Qué elementos del lenguaje de programación te permitieron implementar expresiones regulares?, ¿qué condiciones o recomendaciones se deben seguir al escribir las expresiones regulares?**

  Se utilizaron las funciones del lenguaje que permiten comparar una cadena con un patrón. Para escribir bien las expresiones regulares es importante que los patrones sean claros, que no se mezclen reglas, y que se prueben con diferentes ejemplos para verificar que realmente están funcionando como se espera.

Conclusiones

En esta práctica se logró desarrollar un programa en un lenguaje de programación de alto nivel que permite analizar cadenas de símbolos e identificar diferentes tipos de palabras, como números enteros, palabras en minúsculas, palabras en mayúsculas, identificadores y símbolos. Esto permitió comprender mejor cómo se puede procesar texto dentro de un programa.

El uso de expresiones regulares facilitó la identificación de patrones dentro de las cadenas, ya que permitió clasificar cada palabra de forma automática. Además, la implementación de estructuras de datos simples ayudó a organizar la información obtenida durante el análisis.

Finalmente, la práctica permitió reforzar los conceptos básicos relacionados con lenguajes formales, análisis de cadenas y el uso de expresiones regulares, así como mejorar la comprensión del funcionamiento de un programa que procesa texto de manera estructurada.

Referencias

References

[1] Giró, J., Vázquez, J., Meloni, B., & Constable, L. (2015). *Lenguajes formales y teoría de autómatas*. Editorial Alfaomega, Argentina.

[2] Ruiz Catalán, J. (2010). *Compiladores: teoría e implementación*. Editorial Alfaomega, México.

[3] Brookshear, J. G. (1995). *Teoría de la Computación: Lenguajes formales, autómatas y complejidad*. Editorial Addison-Wesley Iberoamericana, Primera edición, U.S.A.